

Алгоритмы, основанные на обходах

Алгоритмы сами по себе ничего не дают, но они очень важны, потому что они являются базой для огромного количества других алгоритмов. Перечислим некоторые из них: поиск (подсчет) компонент связности (неориентированный граф), поиск кратчайших путей (невзвешенный граф), построение остовного дерева/каркаса (невзвешенный граф), проверка графа на ацикличность, топологическая сортировка, поиск множества фундаментальных циклов, проверка графа на двудольность, поиск точек сочленения/мостов, проверка сильной связности.

Подсчет компонент связности

Мы говорили о том что любой обход нужно запускать в таком цикле:

```
for (v in V)
    if (!used[v])
        dfs(v);
```

Каждый раз когда условие выполняется мы сталкиваемся с очередной компонентой связности, для подсчета просто добавляем счетчик.

```
k = 0;
for (v in V)
    if (!used[v])
        k++;
        dfs(v);
```

Можно использовать любой обход.

Построение кратчайшего пути (невзвешенный граф)

Заметим сразу, что для того чтобы найти кратчайший путь нужно использовать только обход в ширину. Для того чтобы его использовать нужно иметь массив предков.

```
void bfs(v)
    used[v] = 1
    π[u] ← NULL
    push(q, v)
    while (!empty(q))
        pop(q, u)
        for (w in Adj[u])
```

```

if (!used[w])
    used[w] = 1
     $\pi[w] = u$ 
    push(q, w)

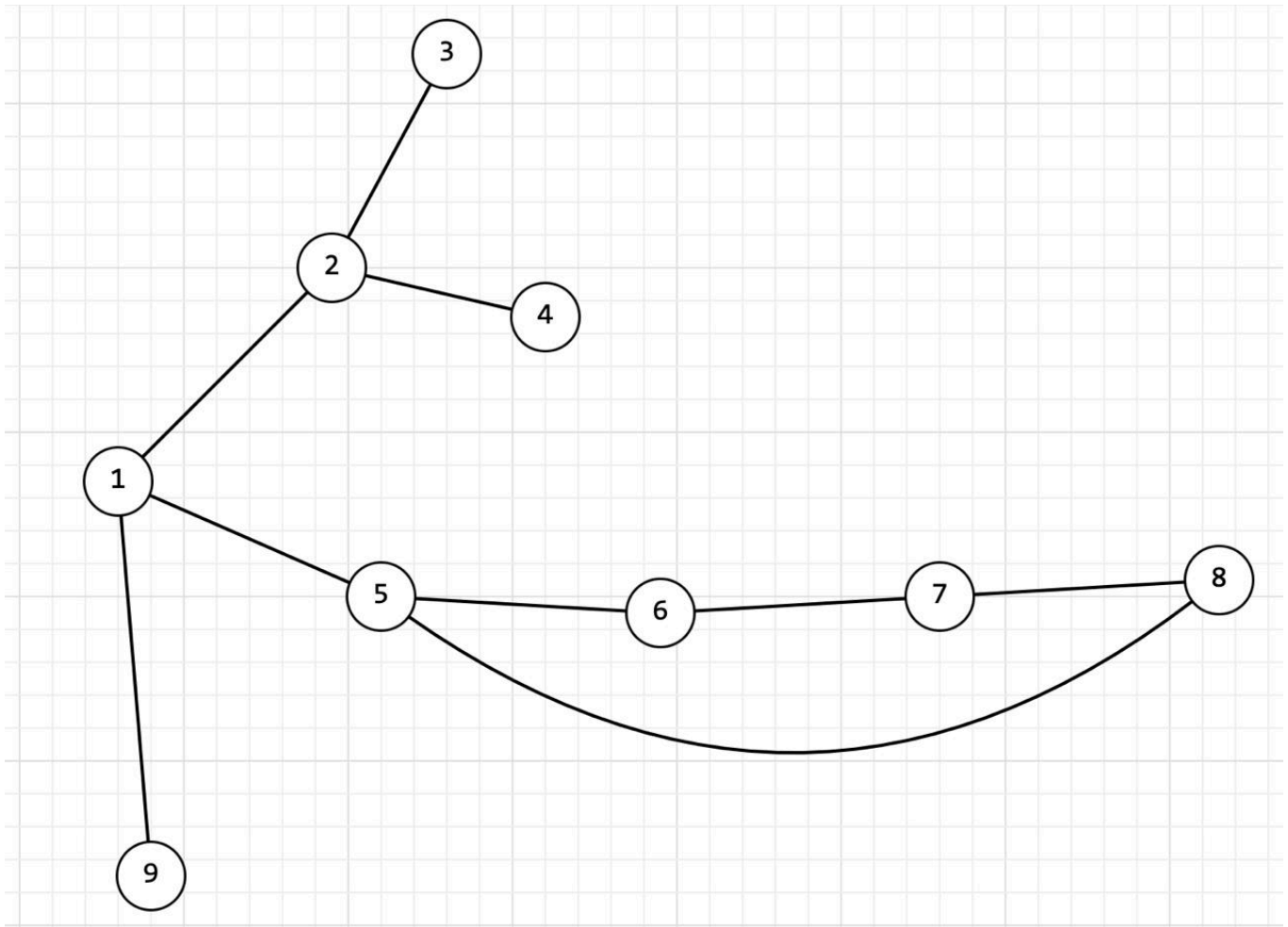
```

Чтобы получить путь нужно выполнить некие действия:

```

print_path(G, s, v)
    if v == s
        then print a
    else if  $\pi[v] == \text{NULL}$ 
        then print "Пусть из s в v отсутствует"
    else print_path(G, s,  $\pi[v]$ )
        print v

```



u	1	2	3	4	5	6	7	8	9
$\pi[u]$	null	1	2	2	1	5	6	5	1

Путь кратчайший: .

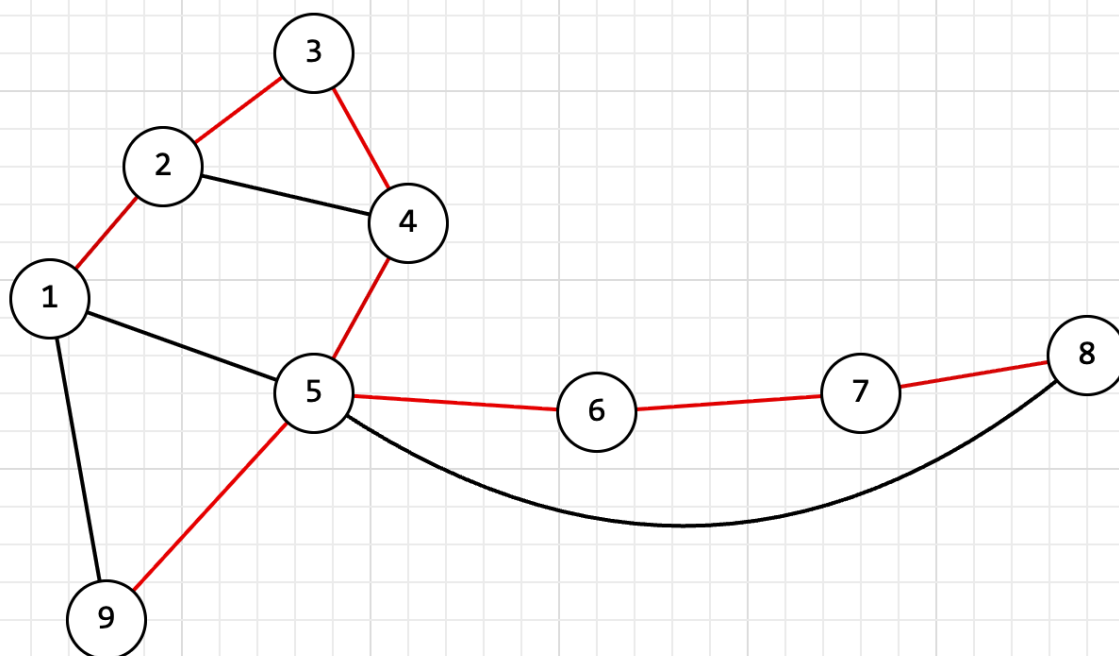
Построение остоного дерева/каркаса (невзвешенный граф)

Дерево – это связный граф, не содержащий циклов. Если остоной подграф является дерево, то он является каркасом.

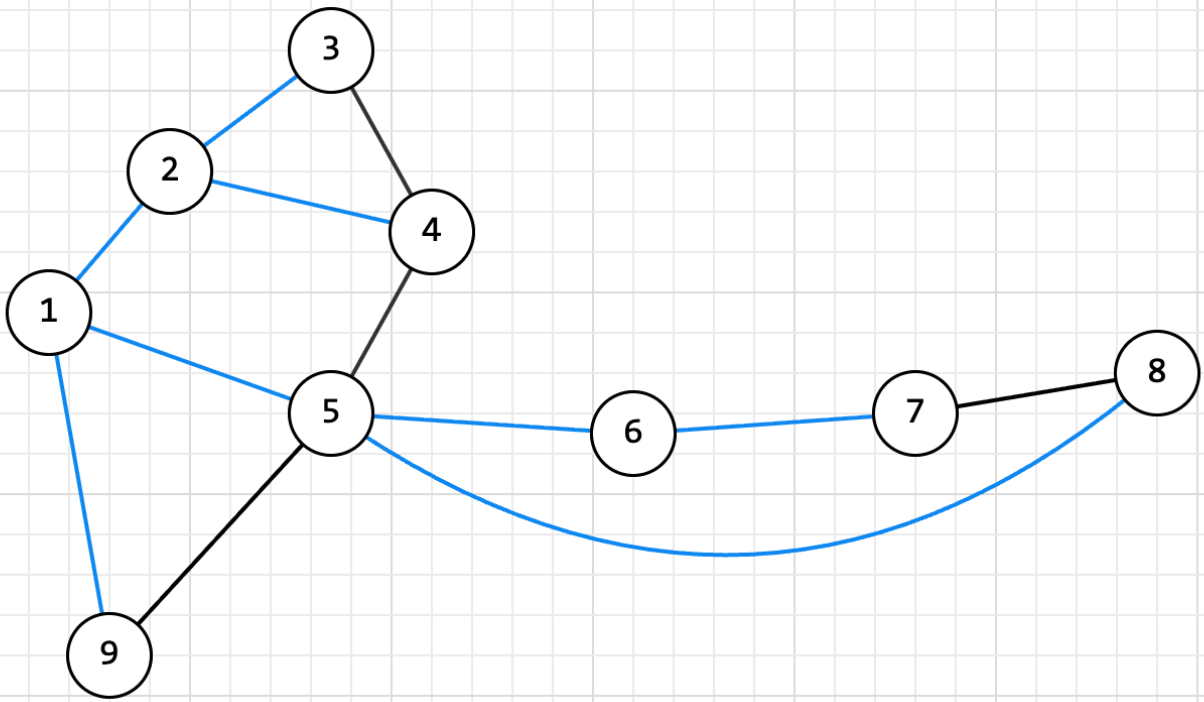
) – дерево

Подходит любой алгоритм обхода, но каркасы будут разные.

Обход в глубину:



Обход в ширину:



Топологическая сортировка

Топологическая сортировка (topological sort) ориентированного ациклического графа (directed acyclic graph – "dag") представляет собой упорядочивание вершин таким образом, что для любой дуги номер вершины меньше номера вершины .

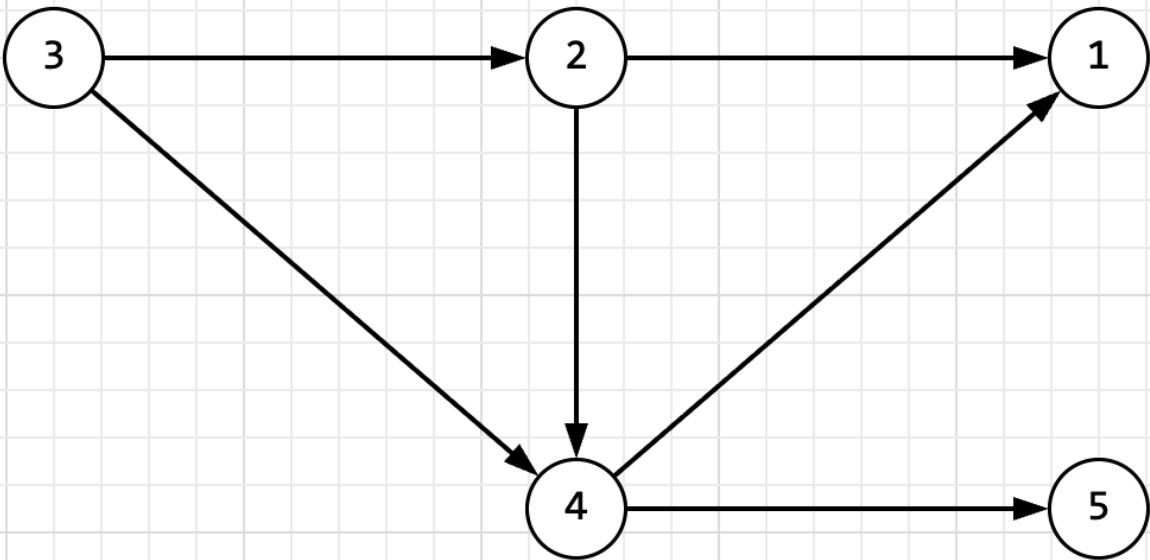
```
topological_sort(G)
```

Вызов DFS(G) для вычисления времени завершения $f[v]$

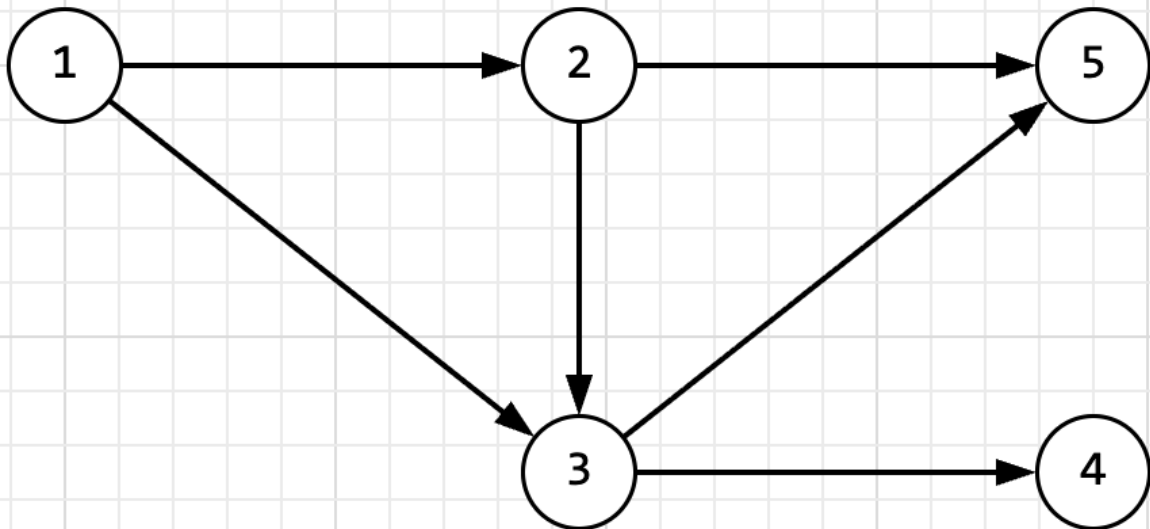
для каждой вершины v

По завершении работы над вершиной внести ее в начало связанного списка

return Связанный список вершин



Обход в глубину из 3 вершины: 1, 5, 4, 2, 3



Д/з

1. По рисунку нарисовать каркас (обходом в глубину и обходом в ширину).

